

7

Strings in Python

7.1 INTRODUCTION

After going through Conditional Constructs in the previous chapter, we will now discuss Strings—an important sequence/collection provided by Python.

Think about beads or anything you might string together. You place **one** bead at a time on to the string. Eventually, you end up with some type of ornamentation—perhaps a necklace or a tree garland. But the point is that these items are made up of individual beads.

The same concept used for a necklace made up of beads holds true for strings in computer. When you see a sentence, you **understand** that it is made up of individual characters that are strung



Fig. 7.1: String in Real World

together by the programming language you use. The language creates a structure that holds individual characters together. So, the language, and not the computer, knows that so many alphabets or numbers in a row (each number being represented as a character) make a string or a sentence. In Python too, like in real life, we deal with strings.

Now the question arises—

- **Where have we seen strings before?**

Recall the simple programs that we have discussed in the previous chapters.

A program to display a "Hello message" to the user that combines with his/her name. Consider the Python code for this:

```
prog_str1.py - C:/Users/preeti/AppData/Local/Programs/Python/Python36-32/Python36-32/Python36-32/py.exe
File Edit Format Run Options Window Help
print("Enter your name: ")
name = input()
print("Hello "+name)
```

```
Python 3.6.5 (v3.6.5:f53c0932b0, Mar 28 2018, 1
6:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for
more information.
>>>
RESTART: C:/Users/preeti/AppData/Local/Program
s/Python/Python36-32/Python36-32/py.exe
Enter your name:
Rinku
HelloRinku
>>>
```

In the above example, the sentence we have typed in `print()` method, i.e., "Enter your name", is a string. Even though you see a sentence, Python sees a specific number of characters. Similarly, the output message displayed as "HelloRinku" is again a string. So, we can say a string more or less behaves like a variable holding characters or alphabets inside double quotes.

7.2 WHAT ARE STRINGS

In Python, a consecutive sequence of characters, which are enclosed or surrounded by single (' ') or double (" ") quotes, is known as a **string**. This sequence of UNICODE characters may include a letter, a number, special character, whitespace or a backslash.

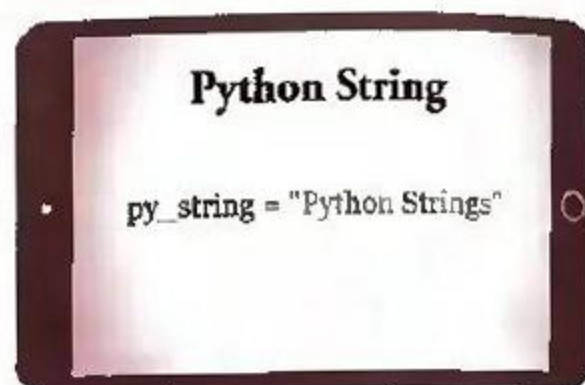
A string may be specified by placing the member characters of the sequence within quotes (single, double or triple). Python treats single quotes the same as double quotes.

Triple quotes are typically used for strings that span multiple lines.

For example,

```
Fruits = 'Mango Apple Grapes'
```

In the above example, we have a string with the names of fruits and space in between, viz. Mango#Apple#Grapes (where # indicates a space) enclosed inside single quotes ' ', thus representing a string value assigned to the variable 'Fruits'. Like other integer variables, strings are also declared in the same manner using assignment(=) operator.



7.3 CREATING STRINGS

Strings are among the most popular data types in Python. We can create them by simply enclosing characters in quotes (single, double or triple). Python treats single quotes the same as double quotes. Sometimes, triple quotes can be used in Python but they are generally used to represent multiline strings and doc strings. An empty string is a string that has 0 character. Creating strings is as simple as assigning a value to a variable.

For example,

```
str1 = "Hello World"
str2 = "Python Programming"
```

```

Python 3.9.0 Shell
File Edit Shell Debug Options Window Help
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct 5 2020, 15:34:40) [MSC
v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more in
formation.
>>> str1="Hello World"
>>> str1
'Hello World'
>>> str2='Python Programming'
>>> str2
'Python Programming'
>>>

```

Text can be enclosed within single or double quotes; both are treated as strings.

Ln: 9 Col: 4

Fig. 7.2: Creating a String

CTM: Text can be enclosed within single or double quotes; both represent a string.

Python does not support a character type; these are treated as strings of length one, thus, also considered a substring.

For example, 'a', 'b', 'c' are all strings of length one.

As we know, double quotes and single quotes are used to represent string in Python language and both are interchangeable. But when a string contains special characters such as quotes, we need to escape them (i.e., with a backslash '\ ' character in front of them to indicate the special nature of the character). The different ways to include quotes inside a string are:

- (i) Escape the single quote by placing a backslash before it or use double quotes instead of single quotes as the enclosing quotes.

```

>>> a="This is Meera\'s pen."
>>> print(a)
This is Meera's pen.
>>> b="Write an article on 'AI' briefly."
>>> print(b)
Write an article on 'AI' briefly.

```

- (ii) If the string contains double quotes, we can use single quotes to represent the string and vice versa so that we don't have to escape the quotes.

```

>>> c='Write an article on "AI" briefly.'
>>> print(c)
Write an article on "AI" briefly.

```

- (iii) If there are both single and double quotes in the string, we need to escape the quotes otherwise it results in a syntactical error.

```

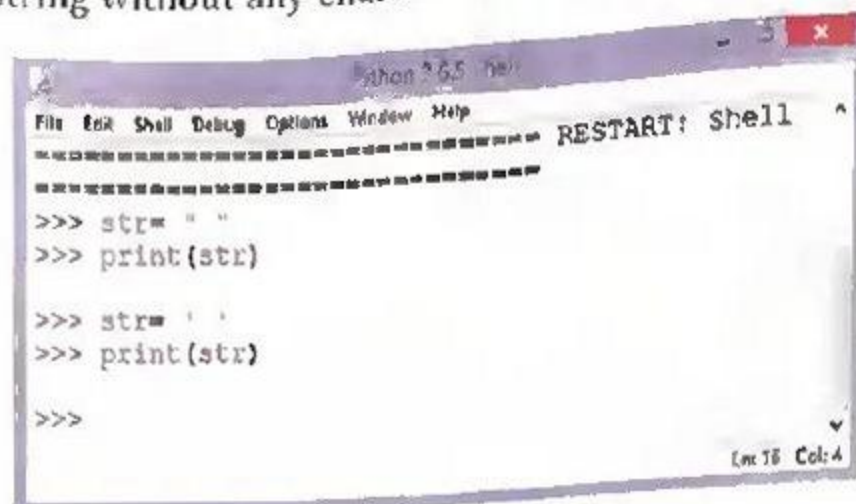
>>> d="She said, 'I'll call you.'"
SyntaxError: invalid syntax
>>> d='She said, "I\'ll call you."'
>>> print(d)
She said, "I'll call you."
>>>

```

CTM: An escape sequence character is represented as a string with one byte of memory.

➤ Empty String

An empty string is a string without any characters inside, having length zero.



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
RESTART: Shell

>>> str = ""
>>> print(str)

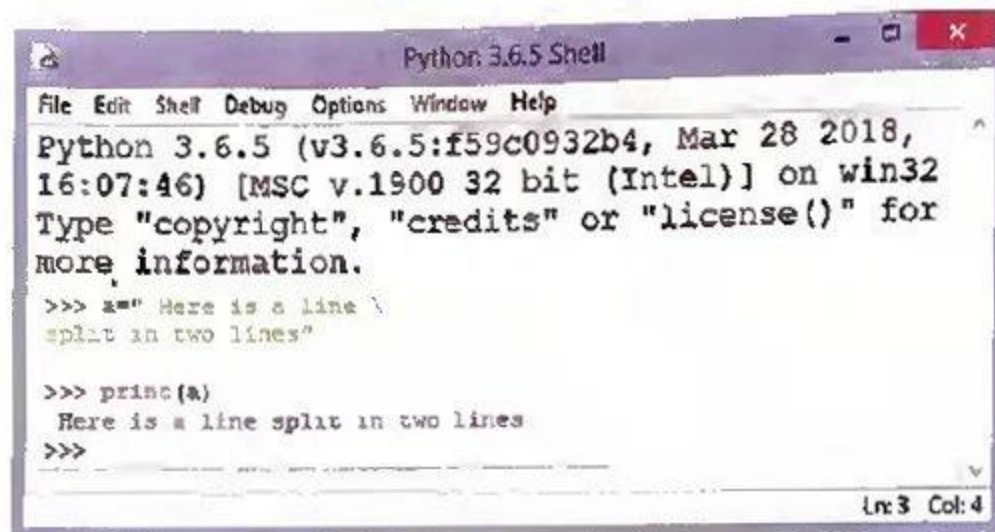
>>> str = ' '
>>> print(str)

>>>
```

As is evident from the above example, string `str` does not hold any value, i.e., it is an empty string. Therefore, when we print an empty string, a blank space gets displayed in the next line.

If the statement is very long, then it can be shifted to multiple lines with the line continuation character (`\`). But it shows the result in the same line.

For example,



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.

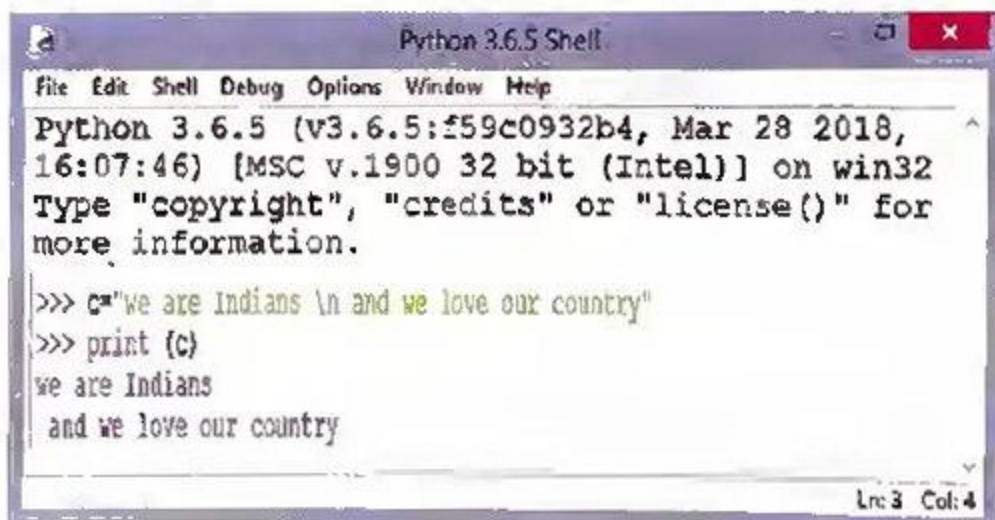
>>> a = "Here is a line \
split in two lines"

>>> print(a)
Here is a line split in two lines

>>>
```

If you wish to display the output on the next line, then the use of escape sequence `\n` becomes mandatory.

For example,

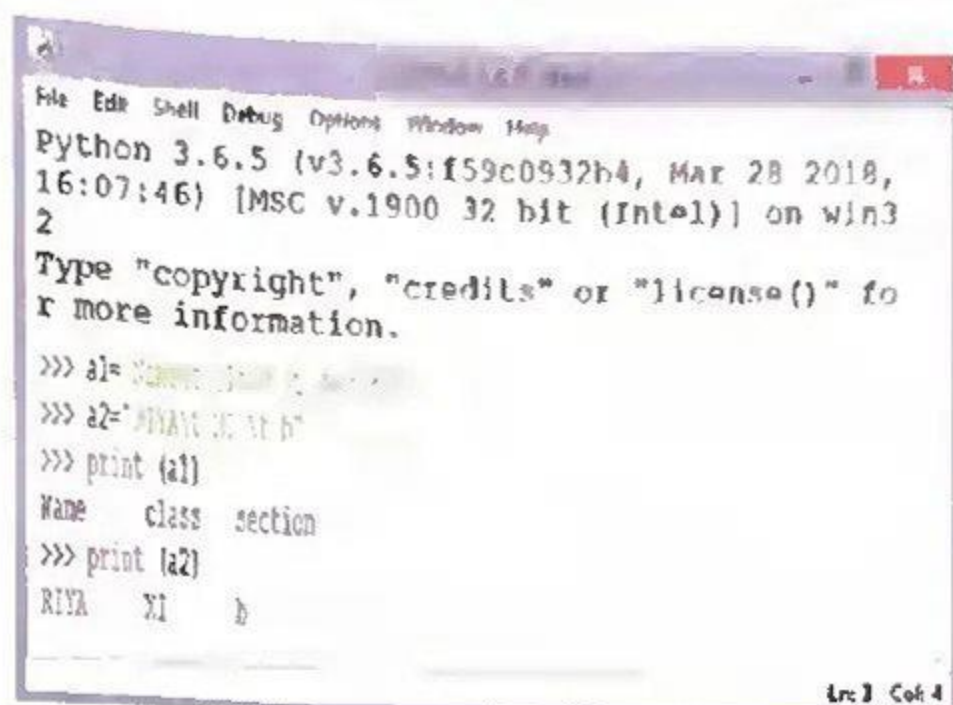


```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.

>>> c = "we are Indians \n and we love our country"
>>> print(c)
we are Indians
and we love our country

>>>
```

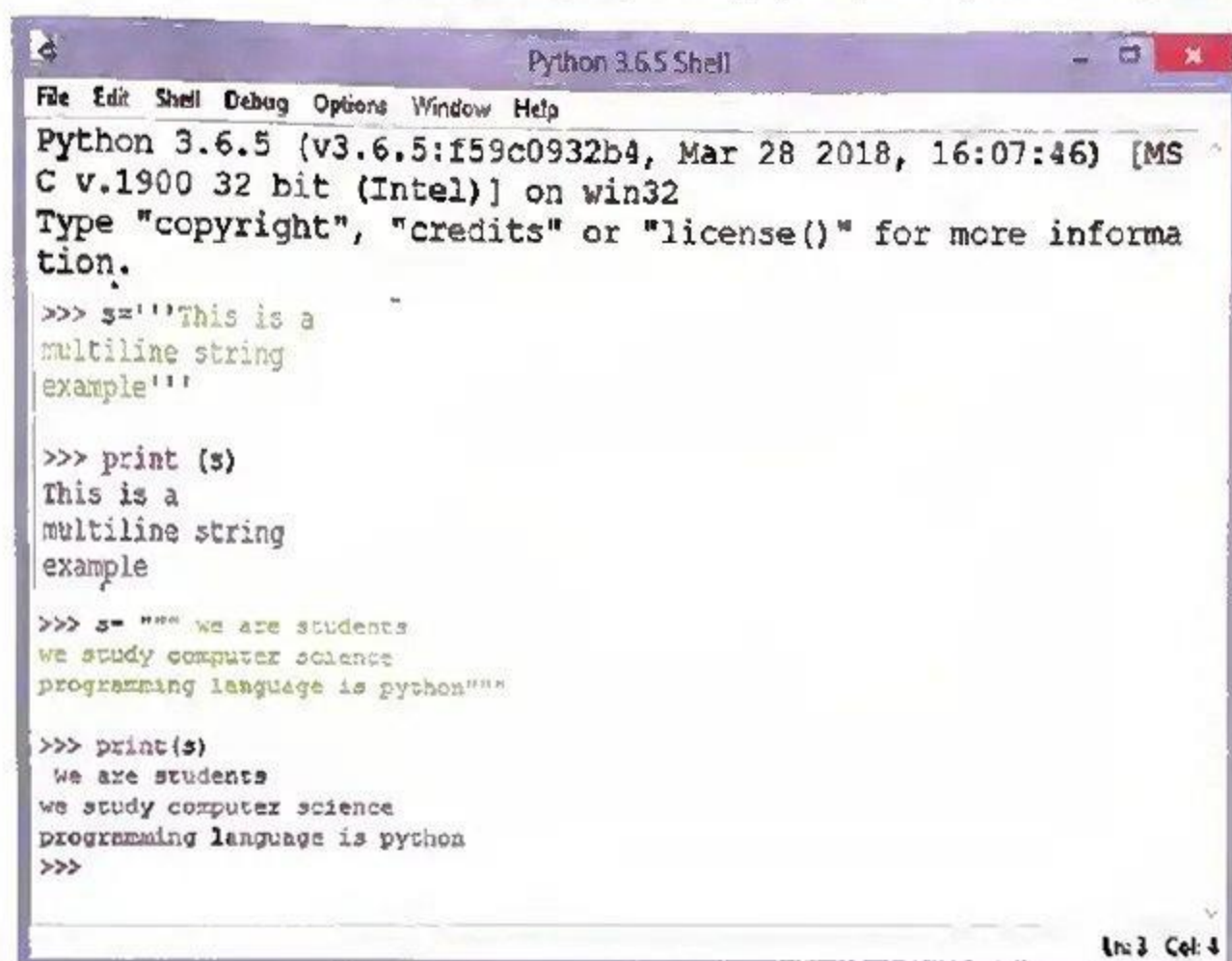
We can also use escape sequence (`\t`) to provide horizontal space in the output. For example,



```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a1= "Name: Riya"
>>> a2= "class: XI"
>>> print(a1)
Name      class      section
>>> print(a2)
Riya      XI          b
```

➤ Multiline Strings

Multiline strings are represented using triple single (`''' '''`) or triple double (`""" """`) quotes.



```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> s= '''This is a
multiline string
example'''
>>> print(s)
This is a
multiline string
example
>>> s= """ we are students
we study computer science
programming language is python"""
>>> print(s)
we are students
we study computer science
programming language is python
>>>
```

CTM: A string is a sequence of characters. We can create them simply by enclosing characters in quotes (single, double or triple).

7.4 ACCESSING CHARACTERS (INDEXING) IN A STRING

We can access individual characters of the string by using the subscript or index value, which is known as indexing.

Each character in a string is located in different slots (like Lockers) as shown in Fig. 7.3 (b).

For example,

Consider a string as:

```
str = "hello world!"
```

h	e	l	l	o		w	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11

Fig. 7.3 (a): Character Representation in a String



Fig. 7.3(b): Lockers

Each character in a string has an **index** value. Indexing starts at 0 and must be an integer. The index operator (square bracket []) retrieves an individual element/character in a string by specifying its position or index value as shown in the output window given below.

```

Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
r more information.
>>> var1 = "Hello World"
>>> print(var1[0])
H
>>> print(var1[4])
o
>>> print(var1[-8])
l
>>> print(var1[-6])

>>> print(var1[-5])
W
>>>
Ln: 14 Col: 4
    
```

We can't use float or other types for index as this will result in **TypeError**. Trying to access a character out of index range will raise an **IndexError**.

We can start index value either from the left or from the right. Left or positive index starts from 0 and ends with l-1 where l is the length of the string and right or negative index starts from -1 to length of the string.

For example,

```
var1 = 'Hello World!'
```

Positive Index	0	1	2	3	4	5	6	7	8	9	10	11
var1	H	e	l	l	o		W	o	r	l	d	!
Negative Index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

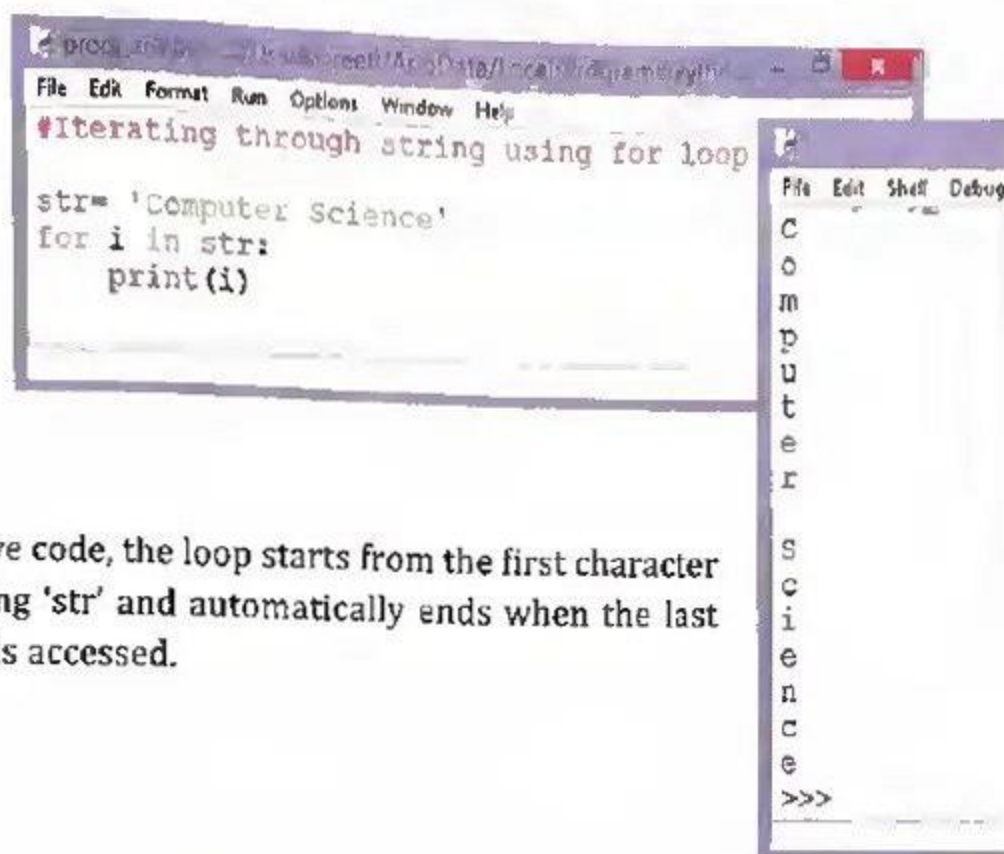
Here, each element or character of the string is associated with an index and can be accessed by its index value. To access any subscript of a string, we use the square brackets along with the index.

7.5 TRAVERSING A STRING

Traversing a string means accessing each character one after the other by iterating through the elements of a string using either *for* or *while* loop.

(a) Iterating through string using *for* loop

Each character of the string can be accessed sequentially using *for* loop. This is done using the indexes, and iterating over it results in displaying the elements of a string, one character at a time.



```
prog_str.py - C:/Users/preeti/AppData/Local/Programs/Python/Python39-64/Python.exe
File Edit Format Run Options Window Help
#Iterating through string using for loop

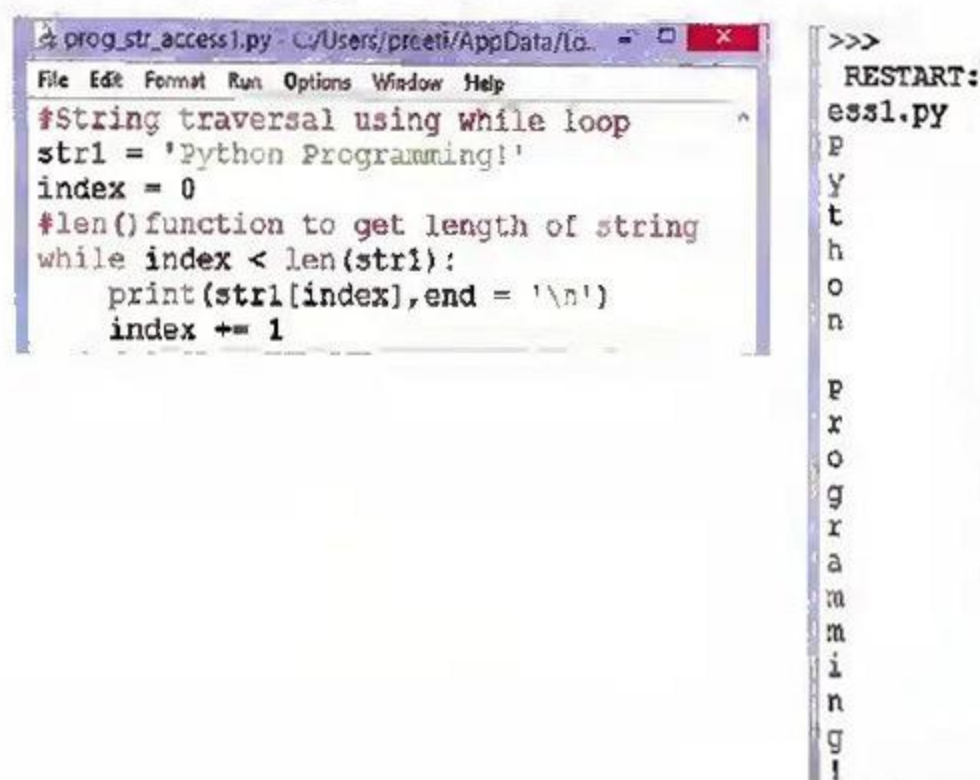
str= 'Computer Science'
for i in str:
    print(i)
```

The output window shows the characters of the string 'Computer Science' printed one by one, each on a new line: C, o, m, p, u, t, e, r, S, c, i, e, n, c, e. The prompt >>> is visible at the bottom of the output window.

In the above code, the loop starts from the first character of the string 'str' and automatically ends when the last character is accessed.

(b) Iterating through string using *while* loop

Like *for*, *while* loop can also be used for string traversal. Each element of the string is accessed one by one and *while* loop iterates till the end of the inputted string, which can be determined using standard library function, `len()` function.



```
prog_str_access1.py - C:/Users/preeti/AppData/Local/Programs/Python/Python39-64/Python.exe
File Edit Format Run Options Window Help
#String traversal using while loop
str1 = 'Python Programming!'
index = 0
#len() function to get length of string
while index < len(str1):
    print(str1[index], end = '\n')
    index += 1
```

The output window shows the string 'Python Programming!' printed character by character, each on a new line: P, y, t, h, o, n, P, r, o, g, r, a, m, m, i, n, g, !. The prompt >>> is visible at the bottom of the output window.

Here, *while* loop runs till the condition `index < len(str)` is True, where index increments from 0 to `len(str1)-1`.

Practical Implementation-1

Write a program to count the number of occurrences of a character in an inputted string.
(Implementing String Traversal)

```
#Function to count the number of times a character occurs in a
#string
str = input("Enter a string: ")
ch = input("Enter the character to be searched: ")
count = 0
for character in str:
    if character == ch:
        count += 1
print("Number of times character",ch,"occurs in the string \
is:",count)
```

```
>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python
py
Enter a string: python is easy and Interactive
Enter the character to be searched: e
Number of times character e occurs in the string is: 3
```

Practical Implementation-2

Write a program to input a string from the user and print it in the reverse order without creating a new string.

```
#Program to display string in reverse order
str = input("Enter a string: ")
for i in range(-1,-len(str)-1,-1):
    print(str[i],end='')
```

```
>>>
RESTART: C:/Users/preeti/AppData
_str.py
Enter a string: Reversing String
gnirts gnisreveR
```

7.6 SPECIAL STRING OPERATIONS

String can be manipulated using operators like concatenate (+), repetition (*) and membership operator like in and not in.

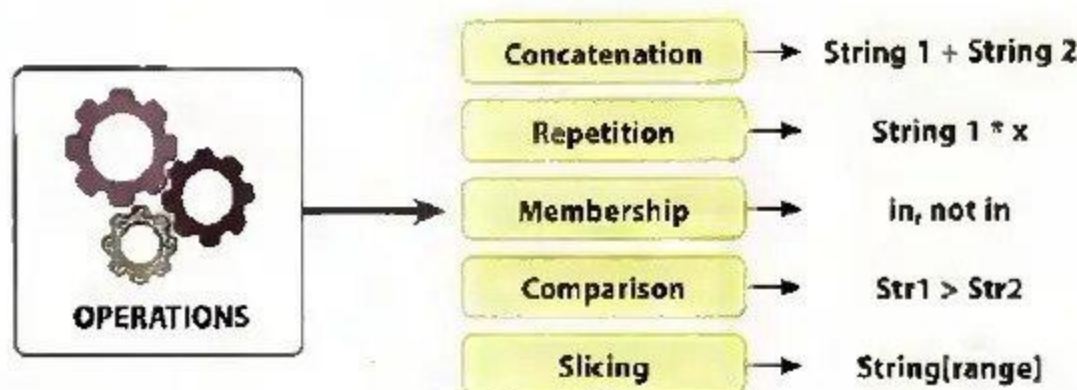


Fig. 7.4: Common String Operations

7.6.1 Concatenating Strings

Concatenation refers to creating a new string by adding two strings. The + operator joins or concatenates strings written on both sides of the operator and creates a new string.

CTM: To concatenate means to join.

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1
900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.

>>> 'Hello ' + 'World'
'Hello World'

>>> c="Good " "Morning"
>>> print(c)
Good Morning
```

If we try to add an integer value to a string, it will generate an error, i.e., `TypeError` will be generated by Python interpreter in case of incompatible data types as shown below:

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1
900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information
>>> 2 + 'book'
Traceback (most recent call last):
  File "<pyshell#16>", line 1, in <module>
    2 + 'book'
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

CTM: You cannot add 'str' and 'int' objects. To use '+' operator, data types should be the same.

However, you can use concatenation operator '+' with numbers and strings separately for performing addition and concatenation operation respectively.

For example,

`6 + 3 = 9` #addition

`'6' + '3' = '63'` #concatenation

But the expression—

`'6' + 3` shall generate an error as shown in the above output window.

7.6.2 Replicating Strings

The * operator creates a new string by repeating multiple copies of the same string.

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
>>> 3*'Hello '
'Hello Hello Hello '
>>> 2*'2'
'22'
```

Using replication operator *, Python creates a new string that is a number of repetitions of the input string.

For example,

```
>>> str= "Python"
>>> str * 3 #using variable str
'PythonPythonPython'
>>> '3' * '5'
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    '3' * '5'
TypeError: can't multiply sequence by non-int of type 'str'
```

CTM: The * operator has to either have both operands of the number types (for performing multiplication) or one string type and one number type (for replication). It cannot work with both operands of string types.

The last statement shall result in an error because strings can't be multiplied; one operand has to be an integer value.

7.6.3 Membership Operators

Python offers two membership operators for checking whether a particular character exists in the given string or not. These operators are 'in' and 'not in'.

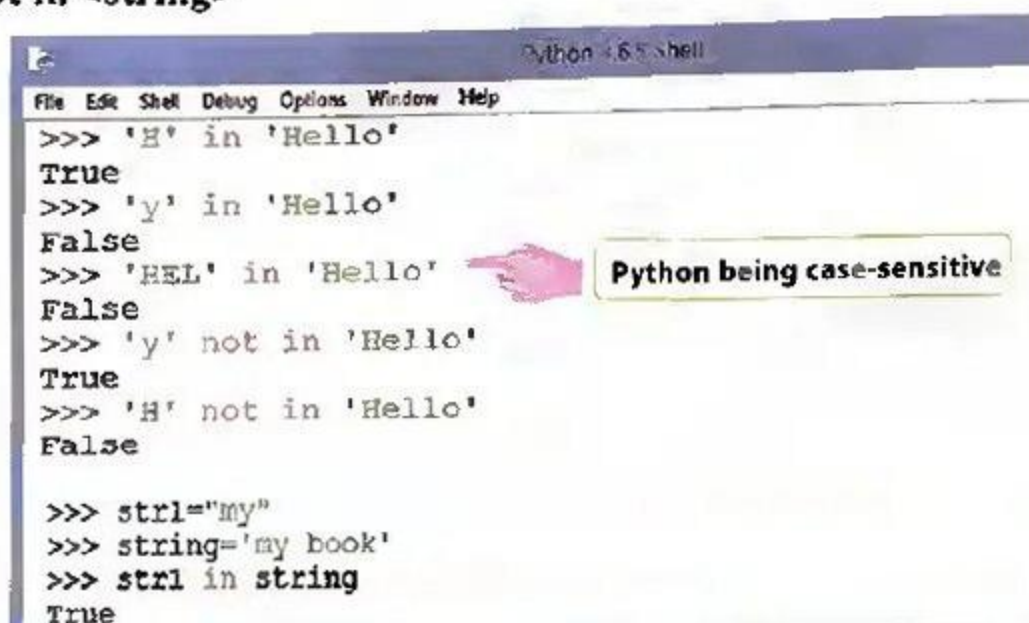
'in' operator: It returns true if a character/substring exists in the given string.

'not in' operator: It returns true if a character/substring does not exist in the given string.

To use membership operator in strings, it is required that both the operands used should be of string type, i.e.,

<substring> in <string>

<substring> not in <string>



```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
>>> 'H' in 'Hello'
True
>>> 'y' in 'Hello'
False
>>> 'HEL' in 'Hello'
False
>>> 'y' not in 'Hello'
True
>>> 'H' not in 'Hello'
False

>>> str1="my"
>>> string='my book'
>>> str1 in string
True
```

A pink arrow points from the text "Python being case-sensitive" to the line `'HEL' in 'Hello'` in the screenshot.

CTM: To use membership operator in strings, it is required that both the operands used should be of string type.

7.6.4 Comparison Operators

You can use relational/comparison operators (>, <, >=, <=, ==, !=) to compare two strings. Python compares strings using ASCII, rather UNICODE value of the characters. Unicode values are also called Ordinal Values.

Python starts by comparing the first element from each sequence (character-by-character comparison). If they are equal, it goes on to the next element, and so on, until it finds elements that differ. Subsequent elements are not considered (even if they are big).

```
>>> str1="Mary"
>>> str2="Mac"
>>> str1<str2
False
```

Here, str1 is "Mary" and str2 is "Mac". The first two characters from str1 and str2 (M and M) are compared. As they are equal, the second two characters (a and a) are compared. Since they are also equal, the third two characters (r and c) are compared. Because 'r' has greater ASCII/Unicode value than 'c', str1 is greater than str2.

```
>>> 'tim' == 'tie'
False
>>> "free" != "freedom"
True
>>> "arrow" > "aron"
True
>>> "right" >= "left"
True
>>> "teeth" < "tee"
False
>>> "yellow" <= "fellow"
False
>>> "abc" > ""
True
```

In all the above examples, Python compares using values (called Ordinal Values). For most common characters, the ASCII and Unicode values are same. The table below gives the most common characters and their ordinal values:

Table 7.1: Common Characters and their Ordinal Values

Characters	Ordinal Values
'0' to '9'	48 to 57
'A' to 'Z'	65 to 90
'a' to 'z'	97 to 122

CTM: Python compares two strings character by character according to their Unicode values, until it finds elements that differ. Subsequent elements are not considered.

7.6.5 String Slicing

Slicing is used to retrieve a subset of values. A slice of a string is nothing but a substring. This extracted substring is termed as slice. Chunk of characters can be extracted from a string using slice operator with two indices in square brackets separated by colon ([:]). The syntax is:

Syntax:

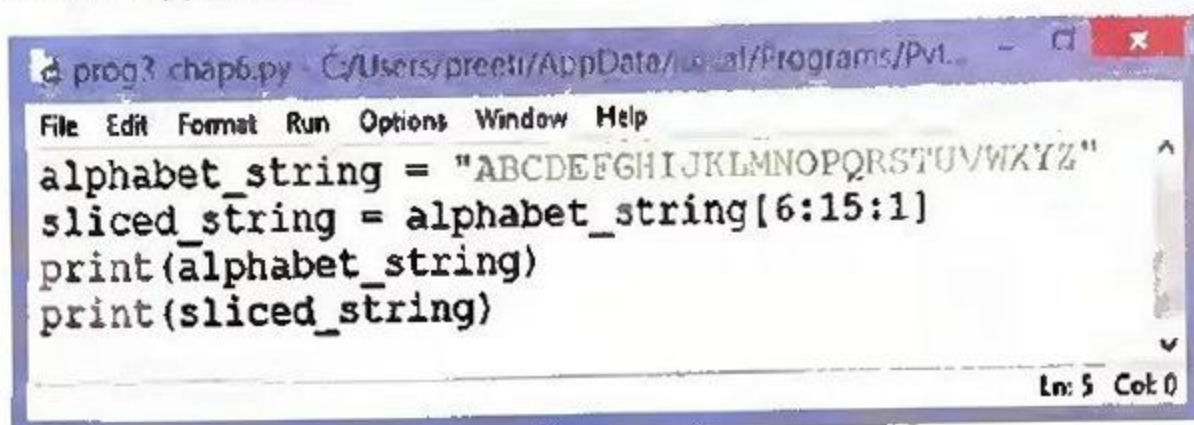
String_name [start:end:step]



Here end is excluded

Python will return a slice of the string by returning characters falling between the starting position till end-1. Let us understand this operation through examples:

Example 1: Consider the given code, the first line of the code shows a string having all the alphabets in the uppercase.



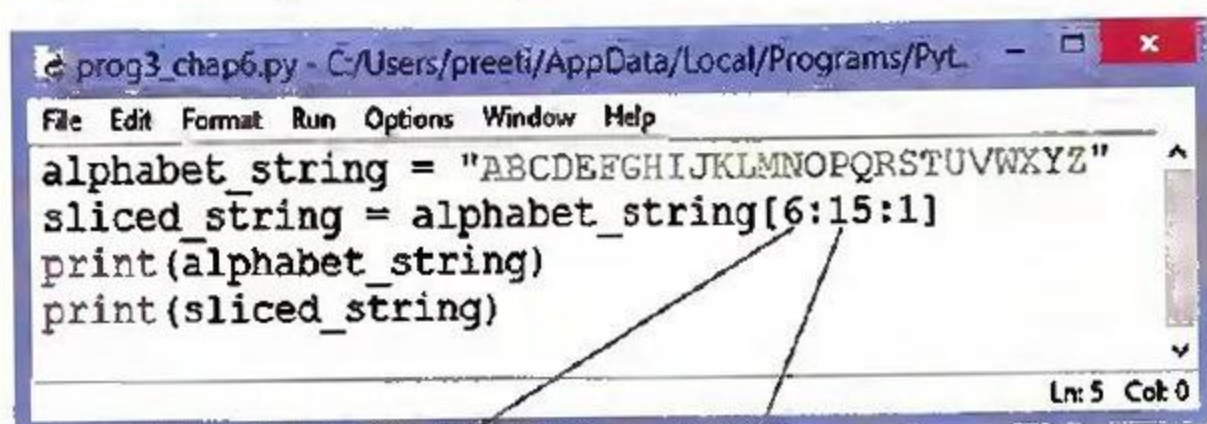
```

prog3_chap6.py - C:/Users/preeti/AppData/Local/Programs/Pyt...
File Edit Format Run Options Window Help
alphabet_string = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
sliced_string = alphabet_string[6:15:1]
print(alphabet_string)
print(sliced_string)
Ln: 5 Col: 0
    
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

alphabet_string

The second statement is going to extract a slice out of this particular string starting from the 6th position and points to 14th position instead of 15th position as shown below.



```

prog3_chap6.py - C:/Users/preeti/AppData/Local/Programs/Pyt...
File Edit Format Run Options Window Help
alphabet_string = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
sliced_string = alphabet_string[6:15:1]
print(alphabet_string)
print(sliced_string)
Ln: 5 Col: 0
    
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

alphabet_string

```

>>>
RESTART: C:/Users/preeti/AppData
py
ABCDEFGHIJKLMNOPQRSTUVWXYZ
GHIJKLMNO
>>>
    
```

Fig. 7.5(a): String Slicing Using Step Value as 1

'end' position is to be excluded. Step 1 ensures that all characters from index position 6th to position 14th are sliced out. Hence, statements 3 and 4 shall give the output as "GHIJKLMNO".

In case the step value is taken as 2 as shown in Fig. 7.5(b), the output shall be obtained by extracting a substring starting from 6th position to 14th position but skipping the adjacent element since step value is 2.

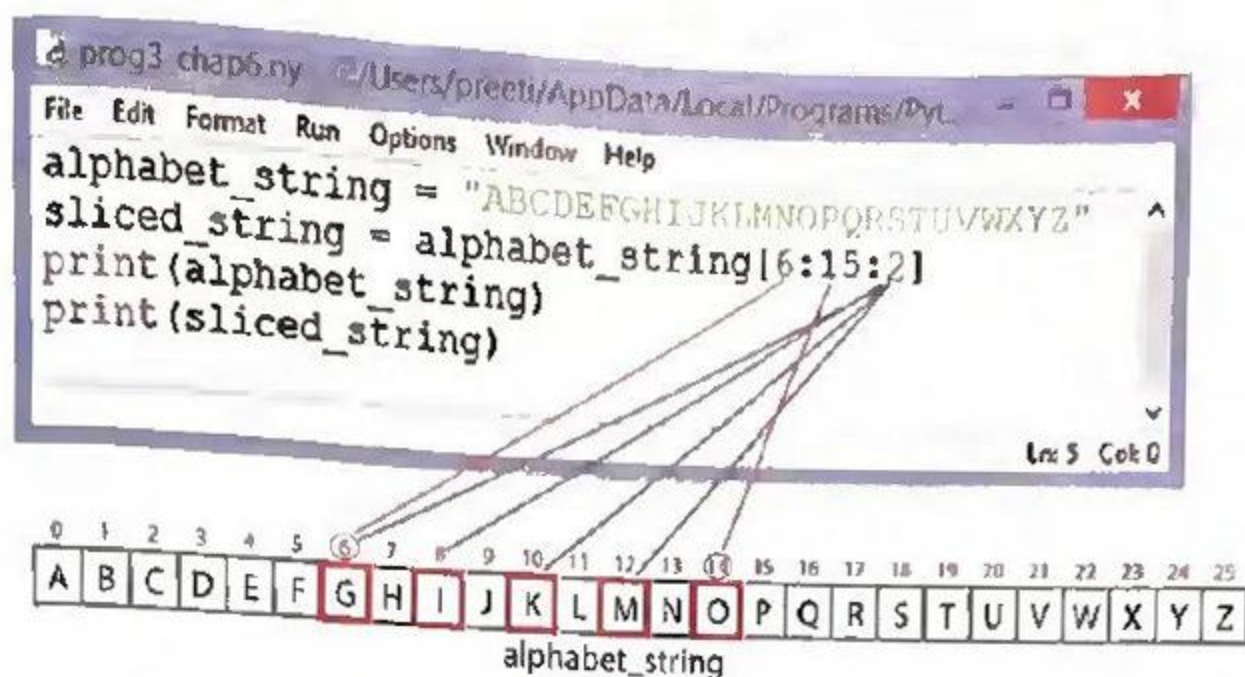


Fig. 7.5(b): String Slicing Using Step Value as 2

Hence, the output displayed is "GIKMO".

Example 2: Consider the given string A = "SAVE MONEY":

String A	S	A	V	E		M	O	N	E	Y
Positive Index	0	1	2	3	4	5	6	7	8	9
Negative Index	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

(a) >>> A = 'SAVE MONEY'

>>> A[1:3]

'AV'

Here, print statement prints the subscript starting from index 1 and ending before index 3, so it will print AV (1st and 2nd index).

(b) >>> A[:3]

'SAV'

Omitting the first index directs the Python interpreter to extract the substring till the second index starting from the beginning, i.e., index 0.

(c) >>> A[3:]

'E MONEY'

Omitting the second index directs the Python interpreter to extract the substring till the end of the string starting from 3rd position.

(d) >>> A[:]

'SAVE MONEY'

Omitting both the indices directs the Python interpreter to extract the entire string starting from 0 till the last index.

(e) >>> A[-2:]

'EY'

For negative indices, the Python interpreter counts from the right side (also shown above). Here, start index is -2, but end index is not given, so it will start printing from index -2 and will print all the characters till the end (-2 starting point till index -1).

(f) `>>> A[:-2]`
`'SAVE MON'`

Omitting the first index directs the Python interpreter to start extracting the substring from the beginning. Since the negative index indicates slicing from the end of the string, the entire string except the last two letters is printed.

(g) `>>> A[::2]`
`'SV OE'`

This will print alternate character from the string.

Practical Implementation-3

Program to input name and print the pattern as given below using string slicing:
(if name is 'ANAND'),

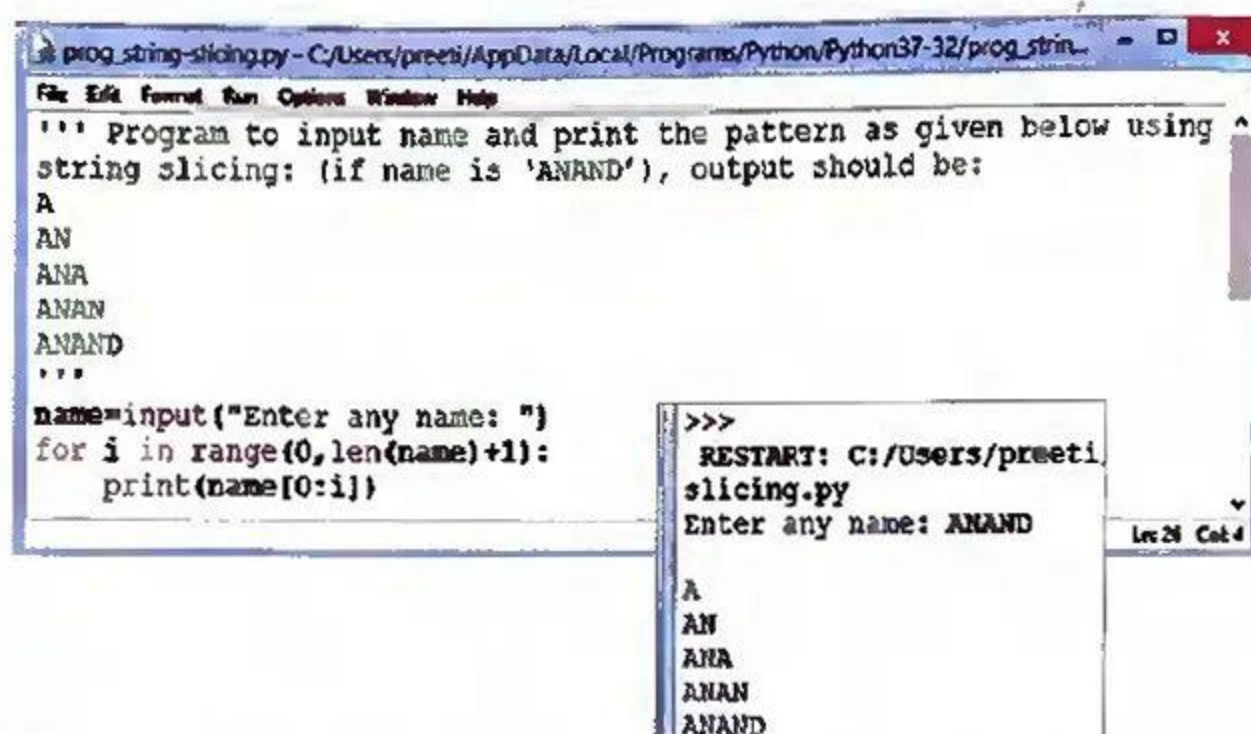
A

AN

ANA

ANAN

ANAND



```
prog_string-slicing.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_strin...
File Edit Format Run Options Window Help
''' Program to input name and print the pattern as given below using
string slicing: (if name is 'ANAND'), output should be:
A
AN
ANA
ANAN
ANAND
'''
name=input("Enter any name: ")
for i in range(0,len(name)+1):
    print(name[0:i])

>>>
RESTART: C:/Users/preeti/Python37-32/Python37-32>
Enter any name: ANAND

A
AN
ANA
ANAN
ANAND
```

7.7 STRINGS ARE IMMUTABLE

Strings are immutable means that the content of the string cannot be changed after it is created. Let us understand the concept of immutability with the help of an example.

```
>>>str1='Student'
```

```
>>>str1[3]='p'
```

TypeError: 'str' object does not support item assignment.

Python does not allow the programmer to change a character in a string. As shown in the above example, str1 has the value 'Student'. An attempt to replace 'd' in the string by 'p' displays a **TypeError**.

CTM: Strings are immutable, i.e., once created, strings cannot be changed.

7.8 STRING METHODS AND BUILT-IN FUNCTIONS

Python provides several built-in functions associated with the strings. These functions allow us to easily modify and manipulate strings. We can think of functions as being actions that we perform on elements of a string in our code. Built-in functions/string manipulation functions are pre-defined in Python programming language and are readily available for use. The syntax is:

String_Object.functionName()

Let us understand these methods.

1. **len()**—This method returns the length of the string.

Syntax:

```
len(str)
```

Here str is the string.

For example,

```
>>> word='Good Morning'
```

```
>>> len(word)
```

```
12
```

```
>>> str1='This is Meera\'s pen'
```

```
>>> len(str1)
```

```
19
```

Here \ is counted as one character.

2. **capitalize()**—This method returns the exact copy of the string with the first letter in uppercase.

Syntax:

```
str.capitalize()
```

Here str is the string.

For example,

```
>>> str1='welcome'
```

```
>>> str1.capitalize()
```

```
'Welcome'
```

```
>>> str1='Welcome'
```

```
>>> str1.capitalize()
```

```
'Welcome'
```

Note: If the string has its first character as capital, then it returns the original string.

Practical Implementation-4

Write a program to accept a string (a sentence) and return a string having first letter of each word in capital letter.

```
Python 2.7.10 - CHAPTER 7: Strings (3.6.3)
File Edit Format Run Options Window Help
str1=input("Enter a string:")
print ("original string:",str1)
str2=""
x = str1.split()
for a in x:
    str2 += a.capitalize()+" "
print (str2)
```

```
Python 3.6.5 Shell
File Edit Shell Debug Options Window Help
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 16:07:46) [MSC v.1900 32 bit (Intel)] on win32
Type "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\CHAPTER 6\prgm_2.py =====
Enter a string: we are learning python
original string: we are learning python
We Are Learning Python
>>>
```

3. **split()**—The **split()** method breaks up a string at the specified separator and returns a list of substrings.

Syntax:

```
str.split([separator, [maxsplit]])
```

The **split()** method takes a maximum of 2 parameters:

- **separator (optional)**—The separator is a delimiter. The string splits at the specified separator. If the separator is not specified, any whitespace (space, newline, etc.) string is a separator.
- **maxsplit (optional)**—The **maxsplit** defines the maximum number of splits. The default value of **maxsplit** is -1, which means no limit on the number of splits.

For example,

```
===== RESTART: Shell =====
>>> x = 'blue:red:green'
>>> x.split(",")
['blue', 'red', 'green']
>>> text = 'Love your country'
>>> print(text.split())
['Love', 'your', 'country']
>>>
```

Here, separator is not specified, so by default, space is a string separator.

The second argument '**maxsplit**' is optional and its default value is zero. If an integer value '**n**' is given for the second argument, the string is split into '**n+1**' strings.

```
===== RESTART: Shell =====
>>> grocery = 'Red:Blue:Orange:Pink'
>>> print(grocery.split(':', 2))
['Red', 'Blue', 'Orange:Pink']
>>> print(grocery.split(':', 1)) #maxsplit is 1
['Red', 'Blue:Orange:Pink']
>>> print(grocery.split(':', 5))
['Red', 'Blue', 'Orange', 'Pink']
>>> print(grocery.split(':', 0)) #maxsplit is 0, so it will return the string
['Red:Blue:Orange:Pink']
>>>
```

4. **replace()**—This function replaces all the occurrences of the old string with the new string.

Syntax: `str.replace(old, new)`

```
===== RESTART: Shell =====
>>> str1 = "This is a string example"
>>> print(str1.replace("is", "was"))
Thwas was a string example
>>> str2 = 'cold coffee'
>>> print(str2.replace('cold', 'hot'))
hot coffee
>>>
```

5. **find()**—This function is used to search the first occurrence of the substring in the given string. The `find()` method returns the lowest index of the substring if it is found in the given string. If the substring is not found, it returns -1. Its syntax is:

```
str.find(sub, start, end)
```

Here,

sub: is the substring which needs to be searched in the given string

start: starting position where substring is to be checked within the string

end: end position is the index of the last value for specified range

```
>>> word = 'Green revolution'
>>> result = word.find('Green')
>>> print (result)
0
>>> result = word.find('green')
>>> print (result)
-1
>>> result = word.find("e", 4)
>>> print (result)
7
>>> result = word.find("en", 5)
>>> print (result)
-1
>>> result = word.find('o', 11, 14) #finding letter 'o'
>>> print (result) # from 11th to 14th position
-1
```

In the last statement, letter 'o' is to be searched for starting from 11th to 13th position (as index 14 is excluded in search). Since 'o' lies at 14th position, it will result in an unsuccessful search and shall return -1 as the output.

6. **index()**—This function is quite similar to `find()` function as it also searches the first occurrence and returns the lowest index of the substring if it is found in the given string but raises an exception if the substring is not present in the given string.

Syntax:

```
index(substring, start, end)
```

```
>>> str1 = "Hello ITS all about STRINGS!!"
>>> str1.index('Hello')
0
>>> str1.index('Hi')
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    str1.index('Hi')
ValueError: substring not found
>>>
```

7. **isalpha()**—This function checks for alphabets in an inputted string. It returns True if the string contains only letters, otherwise returns False.

Syntax:

```
str.isalpha()
```

```
>>> str = "Good"
>>> print(str.isalpha())
True
```

Returns True as no special character or digit is present in the string.

```
>>> str1="This is a string"
>>> print(str1.isalpha())
False
```

Returns False as the string contains spaces.

```
>>> str1="Working with ...Python!!"
>>> print(str1.isalpha())
False
```

Returns False as the string contains special characters and spaces.

8. **isalnum()**—The **isalnum()** method returns True if all the characters are alphanumeric, i.e., alphabet letters (A–Z, a–z) and numbers (0–9).

Example of characters that are not alphanumeric: [space] ! # % & ? etc.

Syntax:

```
string.isalnum()
```

For example,

```
text1="Python 3.8"
x = text1.isalnum()
print(x)
```

```
>>>
RESTART: C:/Users/preeti:
func.py
False
>>>
```

The above function shall return output as False since the text contains space also.

```
text1="Python38"
x = text1.isalnum()
print(x)
```

```
>>>
RESTART: C:/Users/preeti:
func.py
True
>>>
```

As is evident from the second example, the output returned is True since text includes only text and numbers.

9. **isdigit()**—This function returns True if the string contains only digits, otherwise False.

Syntax: `str.isdigit()`

```
>>> str1 = "123456"
>>> print(str1.isdigit())
True
```

Returns True as the string contains only digits.

```
>>> str1 = "Ram bagged 1st position"
>>> print(str1.isdigit())
False
```

Returns False because apart from digits, the string contains letters and spaces.

10. **title()**—This function returns the string with first letter of every word in the string in uppercase and rest in lowercase.

Syntax: `str.title()`

```
>>> str1 = "hello ITS all about STRINGS!!"
>>> str1.title()
'Hello Its All About strings!!'
>>>
```

Thus, **title()** returns a version of the string where each word is title-cased.

11. **count()**—This function returns number of times substring str occurs in the given string. If we do not give start index and end index then searching starts from index 0 and ends at length of the string.

Syntax: `str.count(substring, start, end)`

```
>>> str1 = 'Hello World! Hello Hello'
>>> str1.count('Hello', 12, 25)
2
>>> str1.count('Hello')
3
```

12. **lower()**—This function converts all the uppercase letters in the string into lowercase.

Syntax: `str.lower()`

```
>>> str1 = "Learning PYTHON"
>>> print(str1.lower())
learning python
```

Converts uppercase letters only to lowercase. If already in lowercase, then it will simply return the string.

```
>>> str1 = "learning python"
>>> print(str1.lower())
learning python
```

13. **islower()**—This function returns True if all the letters in the string are in lowercase.

Syntax:

`str.islower()`

```
>>> str1 = "python"
>>> print(str1.islower())
True
>>> str1 = "Python"
>>> print(str1.islower())
False
```

14. **upper()**—This function converts lowercase letters in the string into uppercase.

Syntax:

`str.upper()`

```
>>> var1 = 'Welcome'
>>> print(var1.upper())
WELCOME
```

If already in uppercase, then it will simply return the string.

```
>>> var1 = 'WELCOME'
>>> print(var1.upper())
WELCOME
```

15. **isupper()**—This function returns True if the string is in uppercase.

Syntax:

`str.isupper()`

```
>>> str1 = "PYTHON"
>>> print(str1.isupper())
True
>>> str1 = "python"
>>> print(str1.isupper())
False
```

10. **lstrip()**—This function returns the string after removing the space(s) from the left of the string.

Syntax:

```
str.lstrip()  
or  
str.lstrip(chars)
```

chars (optional)—a string specifying the set of characters to be removed from the left. All combinations of characters in the *chars* argument are removed from the left of the string until left character of the string mismatches.

```
>>> str1= "  Green Revolution"  
>>> print(str1.lstrip())  
Green Revolution
```

Here no argument is given, hence, it removed all leading whitespaces from the left of the string.

```
>>> str2= "Green Revolution"  
>>> print(str2.lstrip("Gr"))  
een Revolution  
>>> str2= "Green Revolution"  
>>> print(str2.lstrip("rG"))  
een Revolution
```

Here, all elements of the given argument are matched with left of the str2 and, if found, are removed.

Practical Implementation-5

Write a program that reads a line and prints its frequency chart like,

- Number of uppercase letters
- Number of lowercase letters
- Number of alphabets
- Number of digits

```
prog_str_freq.py - C:/Users/preeti/AppData/Local/Programs/Python/Py  
File Edit Format Run Options Window Help  
line = input("Enter a line:")  
lowercount = uppercount = 0  
digicount = alphacount = 0  
for a in line:  
    if a.islower():  
        lowercount += 1  
    elif a.isupper():  
        uppercount += 1  
    elif a.isdigit():  
        digicount += 1  
    if a.isalpha():  
        alphacount += 1  
print("Number of uppercase letters :", uppercount)  
print("Number of lowercase letters:", lowercount)  
print("Number of alphabets:", alphacount)  
print("Number of digits:", digicount)
```

```
>>>
RESTART: C:/Users/preeti/AppData/Local/
q.py
Enter a line: Python for BIG data 2020
Number of uppercase letters : 5
Number of lowercase letters: 11
Number of alphabets: 16
Number of digits: 4
```

17. **rstrip()**—This function removes all the trailing whitespaces from the right of the string.

Syntax:

```
rstrip()
or
str.rstrip(chars)
```

chars (optional)—a string specifying the set of characters to be removed from the right.

All combinations of characters in the *chars* argument are removed from the right of the string until the right character of the string mismatches.

```
>>> str1= "Green Revolution "
>>> print(str1.rstrip())
Green Revolution
```

Here, no argument is given, hence, it removed all leading whitespaces from the right of the string.

```
>>> str1= "Computers"
>>> print(str1.rstrip("rs"))
Compute
```

Here, the letters 'rs' are passed as an argument; it is matched from the right of the string and removed from the right of the str1.

18. **strip()**—This function returns the string after removing the spaces both on the left and the right of the string.

Syntax:

```
str.strip()

>>> str1 = "    Hello ITS all about STRINGS!!    "
>>> str1.strip()
'Hello ITS all about STRINGS!!'
>>>
```

19. **isspace()**—This function returns True if the string contains only whitespace characters, otherwise returns False.

Syntax:

```
str.isspace()

>>> str1= "    "
>>> print(str1.isspace())
True

>>> str1="  Python"
>>> print(str1.isspace())
False
```

20. **istitle()**—The *istitle()* function doesn't take any arguments. It returns True if the string is properly "title-cased", else returns False if the string is not a "title-cased" string or an empty string.

Syntax:

```
str. istitle()
>>> str1= "All Learn Python"
>>> print(str1.istitle())
True
>>> s= "All learn Python"
>>> print(s.istitle())
False
>>> s= "This Is @ Symbol"
>>> print(s.istitle())
True
>>> s= "PYTHON"
>>> print(s.istitle())
False
```

21. **join(sequence)**—This function returns a string in which the string elements have been joined by a string separator.

Syntax:

```
str.join(sequence)
```

sequence—join() takes an argument which is of sequence data type, capable of returning its element one at a time. This method returns a string which is the concatenation of each element of the string and the string separator between each element of the string.

```
>>> str1= '12345'
>>> s= '-'
>>> s.join(str1)
'1-2-3-4-5'
>>> str2= 'abcd'
>>> s= '#'
>>> s.join(str2)
'a#b#c#d'
```

22. **swapcase()**—This function converts and returns all uppercase characters into lowercase and vice versa of the given string. It does not take any argument.

Syntax:

```
str.swapcase()
```

The swapcase() function returns a string with all the cases changed.

```
>>> str1= "Welcome"
>>> str1.swapcase()
'wELCOME'
>>> str2= "PYTHON"
>>> str2.swapcase()
'python'
>>> s= "pYTHON"
>>> s.swapcase()
'PyThOn'
```

23. **partition(Separator)**—Partition function is used to split the given string using the specified separator and return a tuple with three parts:

- substring before the separator;
- separator itself;
- a substring after the separator.

Syntax:

```
str.partition(Separator)
```

Separator: This argument is required to separate a string. If the separator is not found, it returns the string itself followed by two empty strings within parentheses as tuple.

```
>>> str3= 'xyz@gmail.com'
>>> str3.partition('@')
('xyz@gmail.com', '@', '')
```

Here, separator is not found, so returns the string itself, followed by two empty strings.

```
>>> str2= "Hardworkpays"
>>> str2.partition('work')
('Hard', 'work', 'pays')
```

Here, str2 is separated in three parts:

- 1) substring before separator, i.e., 'Hard',
- 2) separator itself, i.e., 'work', and
- 3) substring part after separator, i.e., 'pays'.

```
>>> str4= str2.partition('-')
>>> print(str4)
('Hardworkpays', '', '')
```

```
>>> str5= str3.partition('@')
>>> print(str5)
('xyz', '@', 'gmail.com')
```

24. **endswith()**—This function returns True if the given string ends with the specified substring, else returns False.

Syntax:

```
str.endswith(substr)

>>> a="Artificial Intelligence"
>>> a.endswith('Intelligence')
True
>>> a.endswith('Artificial')
False
```

25. **startswith()**—This function returns True if the given string starts with the specified substring, else returns False.

Syntax:

```
str.startswith(substr)

>>> b="Machine Learning"
>>> b.startswith('Mac')
True
>>> b.startswith('learning')
False
```

Above functions in a Nutshell:

```
>>> "Hello World".upper().lower()
'hello world'
>>> "Hello World".lower().upper()
'HELLO WORLD'
>>> "Hello World".find("Wor",1,6)
-1
>>> "Hello World".find("Wor")
6
>>> "Hello World".isalpha()
False
>>> "Hello World".isalnum()
False
>>> "1234".isdigit()
True
>>> "123GH".isdigit()
False
>>> "Hello World".endswith("World")
True
>>> "Hello World".endswith("rld")
True
>>> "Hello World".endswith("Wor")
False
>>> "Hello World".startswith("Hello")
True
```

7.9 OTHER FUNCTIONS

In internal storage or memory of the computer, the characters are stored in integer value. A specific value is used for a given character and it is based on ASCII code. There are different numbers assigned to capital letters and small letters.

Python provides two functions for character encoding: `ord()` and `chr()`.

- **ord()**—This function returns the ASCII/Unicode (Ordinal) of the character.

```
>>> ch= 'b'
>>> ord(ch)
98
>>> ord('A')
65
```

- **chr()**—This function returns the character represented by the inputted Unicode/ASCII number.

```
>>> chr(97)
'a'
>>> chr(66)
'B'
```



MEMORY BYTES

- A string is a sequence of characters.
- We can create them simply by enclosing characters in quotes (single, double or triple).
- Positive subscript helps in accessing the string from the beginning.
- Negative subscript helps in accessing the string from the end.
- '+' operator joins or concatenates the strings on both sides of the operator.
- The '*' operator creates a new string concatenating multiple copies of the same string.
- Operator 'in' returns true if a character exists in the given string.
- Operator 'not in' returns true if a character does not exist in the given string.
- Range Slice gives the characters from the given range using ':'.
➤ You can use relational operators { > , < , >= , <= , == , != } to compare two strings.
- Python compares two strings character by character according to their ASCII values.
- Strings are immutable means that the content of the string cannot be changed after it has been created.
- There are many built-in functions for working with strings in Python.
- Python provides various string functions like `len()`, `capitalize()`, `find()`, etc.
- `split()` method returns a list of all words in the string using `sep` as the delimiter string.
- `upper()` method returns a copy of the string converted into uppercase characters.
- `isdigit()` method returns True if all the characters in the string are digits, otherwise returns False.
- `count()` function will return the total count of a given element in a string.
- `index()` function returns index position of substring.
- `strip()` function performs both `lstrip()` and `rstrip()` on string.

OBJECTIVE TYPE QUESTIONS

1. Fill in the blanks.

- (a) A string is a of characters.
- (b) subscript helps in accessing the string from the beginning.
- (c) subscript helps in accessing the string from the end.
- (d) '+' operator the strings on both sides of the operator.
- (e) The method returns the lowest index of the substring if it is found in the given string.
- (f) function returns the exact copy of the string with the first letter in uppercase.
- (g) Strings cannot be modified as they are data types.
- (h) The sequential accessing of each of the elements in a string is called string
- (i) membership operator returns True if a character or substring exists in given string.
- (j) is a part of the string containing some contiguous characters from the string.
- (k) find() function returns when substring is not found in the main string.
- (l) method returns the lowest index of the substring if it is found in a given string.
- (m) function is used to count how many times an element has occurred in a list.
- (n) function returns True if the given string starts with the specified substring, else returns False.

2. State whether the following statements are True or False.

- (a) An empty string is a string that has zero number of characters.
- (b) The last index number of string is length-1 or -1.
- (c) "abc" * 2 will give abc*2 as output.
- (d) Multiplication operator (*) replicates the string.
- (e) We can combine two strings with the help of & operator.
- (f) When we compare "A" != "a", it will give True as an output.
- (g) String allows forward and backward type of indexing.
- (h) In Python, asc() function returns corresponding Unicode value of a character.
- (i) Size of "\\\\" is 2.
- (j) Multiple line string created with the help of triple quotes will include end line character in the length.
- (k) A string is a mutable sequence of one or more characters.

3. Multiple Choice Questions (MCQs)

- (a) Which of the following is not a Python legal string operation?
 - (i) 'abc' + 'abc' (ii) 'abc' * 3 (iii) 'abc' + 3 (iv) 'abc'.lower()
- (b) In Python, string + and * represents which of the following operations?
 - (i) Concatenation, Replication (ii) Addition, Multiplication
 - (iii) Neither (a) nor (b) (iv) Both (a) and (b)
- (c) Which of following is not a valid string operation?
 - (i) Slicing (ii) Concatenation (iii) Repetition (iv) Floor
- (d) How many times is the word "Python" printed in the following statement?
s= 'I love Python'
for ch in s[3:8]:
 print('Python')
 - (i) 11 times (ii) 8 times (iii) 3 times (iv) 5 times
- (e) Which of the following is the correct syntax of String Slicing?
 - (i) String_name[start : end] (ii) String_name[start : step]
 - (iii) String_name[step : end] (iv) String_name[step : start]

- (f) What is the correct Python code to display the last four characters of "Digital India"?
- (i) `str[-4:]` (ii) `str[4:]` (iii) `str[*4:]` (iv) `str[-4:]`
- (g) What will be the output of the following code?
- ```
str1= "I love Python."
strlen = len(str1)
print(strlen)
```
- (i) 9                                      (ii) 29                                      (iii) 14                                      (iv) 3
- (h) What will be the output of the following code?
- ```
Str1= 'My name is digital'
Str2=Str1[3:7]
strlen = len(Str2)
print(strlen)
```
- (i) 4 (ii) 14 (iii) 24 (iv) 34
- (i) Which method removes all the trailing whitespaces from the right of the string?
- (i) `tolower()` (ii) `upper()` (iii) `lstrip()` (iv) `rstrip()`
- (j) To concatenate means to
- (i) replicate (ii) join (iii) split (iv) multiply
- (k) function will always return tuple of 3 elements.
- (i) `index()` (ii) `split()` (iii) `partition()` (iv) `strip()`
- (l) What will be the output of the following code?
- ```
A='Virtual Reality'
A.replace('Virtual', 'Augmented')
```
- (i) Virtual Augmented                      (ii) Reality Augmented  
(iii) Augmented Virtual                      (iv) Augmented Reality
- (m) What will be the output of the following?
- ```
print("ComputerScience".split("er",2))
```
- (i) ["Computer", "Science"] (ii) ["Comput", "Science"]
(iii) ["Comput", "erScience"] (iv) ["Comput", "er", "Science"]

SOLVED QUESTIONS

1. What is a string?

Ans. A string is a sequence of characters where each character can be a letter, number or special symbol. We can enclose characters in quotes (single, double or triple).

2. What is slicing in string?

Ans. Slicing returns the substring from the given string using slicing operator [:].

3. What is traversing a string?

Ans. Traversing a string means accessing each character one after the other by iterating through the elements of a string using either *for* or *while* loop.

4. Write the output of the following statement:

```
str1 = 'This is Mohit\'s "CS" book'
print(len(str1))
```

Ans. 25

5. Consider the string `str="Green Revolution"`. Write statements in Python to implement the following:

- To display the last four characters.
- To display the starting index for the substring 'vo'.
- To check whether the string contains 'vol' or not.
- To repeat the string 3 times.

10. Find the output of the following:

```
word = 'green vegetables'
print(word.find('g', 2))
print(word.find('veg', 2))
print(word.find('tab', 4, 15))
print(word.find('eg', 6, 8))
```

Ans. Output:

```
8
6
10
-1
```

11. How can you create an empty string?

Ans. An empty string can be created by using either double quotes ("") or single quotes (' ') and assigning it to a variable.

12. Differentiate between indexing and traversing of string.

Ans. *Indexing:* Accessing individual characters of the string by using the subscript or index value is called indexing. The index specifies the character to be accessed in the string and is written in square brackets []

Traversing: Accessing each character one after the other by iterating through the elements of a string using either *for* or *while* loop.

13. Why is string an immutable data type?

Ans. The contents of the string cannot be changed after it has been created; that is why it is an immutable data type.

For example,

Str1='Sound'

Str1[0]='P'

would result in – TypeError: 'str' object does not support item assignment.

14. A string called *str* contains whitespaces at the left of the string as given.

```
str = "    Python Program"
```

Write the command to delete the spaces.

Ans. The command is: `str.lstrip()`

15. What will be the value stored in *bval* upon execution if two strings *str1* and *str2* are taken as "Delhi" and "New Delhi"?

(i) `bval = str1 > str2`

(ii) `bval = str1.lower() < str2`

Ans. (i) False

(ii) False

16. Write a Python code to access each character, one by one, of string 'hello'.

Ans. `for ch in 'hello':`

```
    print(ch)
```

Output:

```
h
e
l
l
o
```

17. Write a program to count the number of vowels in the string 'pineapple'.

Ans. Code:

```
word = 'pineapple'
count = 0
for letter in word:
    if letter in ('i', 'u', 'a', 'e', 'o'):
        count=count+ 1
print(count)
```

Output:

```
4
```

18. What will be the output of the following program snippet?

```
str = "aNDaRiEl"
nstr = " "
for i in range(len(str)):
    if str[i].isupper():
        nstr = nstr + str[i].lower()
    else:
        nstr = nstr + str[i].upper()
print(nstr)
```

Ans. The output is:

AndARiEl

19. Write the output of the following program:

```
Str="CBSE Digital India"
for i in range(len(Str)-1,0,-1):
    if Str[i].isupper():
        print(Str[i].lower(),end="")
    elif i%2==0:
        if Str[i].islower():
            print(Str[i].upper(),end="")
        else:
            print("@",end="")
```

Ans. INi@AIId@esb

20. Write a program to check whether the string is a palindrome or not.

Ans. Code:

```
str=input("Enter the String:")
l=len(str)
p=l-1
index=0
while(index<p):
    if(str[index]==str[p]):
        index=index+1
        p=p-1
    else:
        print("String is not a palindrome")
        break
else:
    print("String is a Palindrome")
```

21. Write a program that reads a line, then counts words and displays how many words are there in the line.

Ans. Code:

```
line=input("Enter line:")
x=line.split()
cnt=0
for i in x:
    cnt=cnt+1
print(cnt)
```



22. Write a program that reads a string and then prints a string that capitalizes every other letter in the string.
e.g., school becomes sChOoL.

```
Ans. string=input("Enter a string:")
length = len(string)
print("original string:",string)
string2=""
for a in range(0, length):
    if a%2==0:
        string2 += string[a]
    else:
        string2 += string[a].upper()
print(string2)
```

23. Write a program that reads a line, then counts how many times a substring "is" appears in the line and displays the count.

```
Ans. str1=input("Enter line:")
str2="is"
L=str1.split()
cnt=0
for i in L:
    if i==str2:
        cnt+=1
print("Substring is appearing:",cnt)
```

24. Write a function VowCount() in Python which takes a string as an argument; the function should count and display the occurrence of words starting with a vowel in the given string.

Example:

If the string content is as follows:

Updated information is simplified by official websites.

The VowCount() function should display the output as

Updated

information

is

official

Vowel words: 4

```
Ans. def VowCount():
    cnt=0
    str1= input("Enter a string:")
    word=str1.split()
    for i in word:
        if i[0]in 'aeiou' or i[0]in 'AEIOU':
            cnt+=1
            print(i)
    print("Vowelwords:",cnt)
```

25. Write a program that reads a string and displays the longest substring of the given string.

```
Ans. str1=input("Enter a string:")
word=str1.split()
maxlength=0
maxword=""
for i in word:
    x = len(i)
    if x>maxlength and i.isalpha()==True:
        print(i)
        maxlength=x
        maxword=i
print("Substring with maximum length is:",maxword)
```

Output:

Enter a string:Hello Python
 Hello
 Python
 Substring with maximum length is: Python

26. Write a program to remove vowels from a string.

Ans.

```
str1=input("Enter string:")
str2=""
for i in range(len(str1)):
    if str1[i] not in "ieouaAOIUE":
        str2= str2+str1[i]
print("Original string:",str1)
print("New string is:",str2)
```

Output:

Original string : "We are learning python"
 New string is : "W r lrnng pythn"

27. What will be the output of the following program?

```
mes1 = ["SKy"]
mes2 = ["ThE"]
mes3 = ["LiMIT"]
l1 = len(mes1)
l2 = len(mes2)
l3 = len(mes3)
n = l1+l2+l3
for C in range(1,n):
    if (C%4 == 0):
        print(mes2)
        l2 = l2 - 1
    else:
        if (C%3 == 0):
            print(mes1)
            l1 = l1 - 1
        else:
            print(mes3)
            l3 = l3 - 1
```

Ans: ['LiMIT']
 ['LiMIT']

28. Write a program to input a string having some digits and return the sum of digits present in this string.

Ans:

```
a=input("Enter a string with digit:")
c=0
for i in a:
    if i.isdigit():
        c+=int(i)
print(c)
```

29. Find the output of the following code:

```
S="Wow Python"
print(len(s))
print(s[0],s[-1])
```

Ans: 10
 W n

UNSOLVED QUESTIONS

1. How does '*' operator behave on strings?
2. Explain function split() with an example.
3. How many times is the word 'HELLO' printed in the following statement?

```
s="python rocks"
for ch in s:
    print("Hello")
```

4. Write the output of the following:

```
>>> x="hello"
>>> print(x[1:-2])
```

5. Write the output of the following:

```
(a) string = "Hello Madam, I love Tutorials"
    substring = "Madam"
    if string.find(substring) != -1:
        print("Python found the substring!")
    else:
        print("Python did NOT find the substring!")
```

```
(b) s = "Strings in Python"
    print(s.capitalize())
    print(s.title())
    s6=s.replace("in", "data type")
    print(s6)
```

6. Find the output of the following:

```
word = 'work hard'
result = word.find('work')
print("Substring, 'work', found at index:", result)
result = word.find('har')
print("Substring, 'har ', found at index:", result)
if (word.find('pawan') != -1):
    print("Contains given substring ")
else:
    print("Doesn't contain given substring")
```

7. Consider the following string mySubject:

```
mySubject = "Computer Science"
```

What will be the output of the following string operations?

- (i) print(mySubject[0:len(mySubject)])
- (ii) print(mySubject[-7:-1])
- (iii) print(mySubject[:2])
- (iv) print(mySubject[len(mySubject)-1])
- (v) print(2*mySubject)
- (vi) print(mySubject[:-2])
- (vii) print(mySubject[:3] + mySubject[3:])
- (viii) print(mySubject.swapcase())
- (ix) print(mySubject.startswith('Comp'))
- (x) print(mySubject.isalpha())

8. Write the Python statement and the output for the following:

- (a) Find the third occurrence of 'e' in 'sequence'.
- (b) Change the case of each letter in string 'FuNcTiON'.
- (c) Whether 'Z' exists in string 'School' or not.

9. Consider the string `str1="Global Warming"`.
Write statements in Python to implement the following:
 - (a) To display the last four characters.
 - (b) To replace all the occurrences of letter 'a' in the string with "".
10. Write the output of the following program code


```
str = 'virus-virus-virus'
dl = '-'
str = ["Fundamentals", "of", "Python"]
dl = "" #no space
print(dl.join(str))
```
11. What will be the output of the following code?


```
Text = "Mind@Work!"
ln = len(Text)
nText = ""
for i in range(0,ln):
    if Text[i].isupper():
        nText = nText + Text[i].lower()
    elif Text[i].isalpha():
        nText = nText + Text[i].upper()
    else:
        nText = nText + 'A'
print(nText)
```
12. Input the string 'My School'. Write a script to partition the string at the occurrence of letter 'h'.
13. Write a program to convert a string with more than one word into titlecase string where string is passed as parameter. (Titlecase means that the first letter of each word is capitalized.)
14. Write a program that takes a sentence as an input parameter where each word in the sentence is separated by a space. The function should replace each blank with a hyphen and then return the modified sentence.
15. Write a script to partition the string 'INSTITUTE' at the occurrence of letter 'T'.
16. What will be the output of the following programming code?


```
str="My Python Programming"
print(str[:-5:-1])
print(str[1:5])
print(str[:-4])
print(str[0:])
print(str[:13-4])
print(str[:3])
```
17. Write a program to count the number of each vowel in a given string.
18. Write a program that reads a line, then counts how many times the word 'is' appears in the line and displays the count.
19. Write a program to remove 'l' (if any) from a string.

CASE-BASED/SOURCE-BASED INTEGRATED QUESTIONS

1. ABC Dialling Networks Pvt. Ltd. is a company that deals in maintaining the phone numbers of customers globally. It prefixes the area code automatically at the time of adding the phone number in its database.
Write a program that prompts for a phone number of 10 digits and two dashes, a dash each after area code and the next three numbers. For example, 017-555-1212 is a legal input. Display if the phone number entered is a valid format or not and if the phone number is valid and not (i.e., contains just the digits and dashes at specific places).

Ans.

```
prog_string_casestudy1.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_string_casestudy1.py (3.7.0)
File Edit Format Run Options Window Help
#Code for adding phone numbers in the proper format
p = input("Enter Phone Number :")
val = False
# length must be 12
if len(p) == 12 and p[3] == '-' and p[7] == '-':
    if (p[3]+p[4:7]+p[8:]).isdigit(): # all the characters except dashes should be digits
        val = True
if val:
    print(p, "is valid")
else:
    print(p, "is invalid")

>>>
RESTART: C:/Users/preeti/AppData/Local/casestudy1.py
Enter Phone Number :223098888
223098888 is invalid
>>>
RESTART: C:/Users/preeti/AppData/Local/casestudy1.py
Enter Phone Number :989-234-3377
989-234-3377 is valid
```

2. Kids Elementary Technologies help nursery children to improve their handwriting and writing skills such as word formation, recognition of lower and uppercase letters and small sentences formation.

Write a program that should prompt the child to type some sentence(s) followed by "enter". It should then print the original sentence(s) and the following statistics relating to the sentence(s):

- Number of words
- Number of characters (including white-space and punctuation)
- Percentage of characters that are alphanumeric

Ans.

```
prog_sentence_parameters.py - C:/Users/preeti/AppData/Local/Programs/Python/Python37-32/prog_sentence_param...
File Edit Format Run Options Window Help
s = input("Enter a sentence : ")
number_of_words = 1
number_of_characters = len(s)
al_num = 0
for i in s:
    if i.isalnum():
        al_num += 1
    if i == ' ': # there is a space means there is another word
        number_of_words += 1
print("number of words are", number_of_words)
print("number of characters are", number_of_characters)
print("percentage of characters that are alphanumeric is", al_num*100/len(s), "%")

>>>
RESTART: C:/Users/preeti/AppData/Local/Programs/Python/Py...
Enter a sentence : This Utility is for Nursery KIDS
number of words are 6
number of characters are 32
percentage of characters that are alphanumeric is 84.375 %
```